DOCUMENTATION ON DAY 2 TASK

Name: Moturi Lalitha Sowjanya

Branch: CSM-AI&ML (LE1)

Team: 4

Date: 02-06-26

---

1. OBJECTIVE

The objective of this task was to understand the best practices, common mistakes, and limitations of Retrieval-Augmented Generation (RAG) systems. The task also aimed to provide knowledge about embeddings and vector databases, which are important components of modern RAG applications.

---

2. RESEARCH TOPICS

2.1. DO'S OF RAG SYSTEMS

Retrieval-Augmented Generation (RAG) systems improve the quality of AI responses by retrieving relevant information from external sources before generating answers. Following best practices helps improve accuracy, reliability, and overall system performance.

- **Use High-Quality Data**

The knowledge base should contain accurate, verified, and updated information. Since RAG systems generate answers using retrieved documents, poor-quality data can result in inaccurate responses.

- **Perform Proper Document Chunking**

Large documents should be divided into smaller meaningful chunks. Proper chunking improves retrieval accuracy and helps the system locate relevant information efficiently.

- **Use Metadata**

Metadata such as file name, topic, date, and author helps organize documents and improves retrieval precision.

- **Use Strong Embedding Models**

Embedding models convert text into numerical vectors that represent semantic meaning. High-quality embeddings improve retrieval performance.

- **Use Vector Databases**

Vector databases store embeddings and perform similarity searches efficiently. They help retrieve the most relevant information from large document collections.

- **Retrieve Relevant Context**

Only relevant information should be provided to the language model. Excessive or unrelated context may reduce response quality.

- **Provide Source Citations**

Including references to source documents increases transparency and allows users to verify the information.

- **Handle Unknown Questions Safely**

If relevant information is unavailable, the system should indicate that the answer could not be found instead of generating unsupported information.

- **Update the Knowledge Base Regularly**

Documents should be updated whenever information changes to ensure accurate responses.

- **Continuously Evaluate the System**

Regular testing and evaluation help identify retrieval errors and improve overall system performance.

2.2. DON'TS OF RAG SYSTEMS

Certain practices can negatively affect the performance of a RAG system and should be avoided.

- **Don't Use Poor-Quality Documents**

Incorrect, outdated, or duplicate documents may lead to inaccurate responses.

- **Don't Use Extremely Large Chunks**

Very large chunks reduce retrieval accuracy and make it difficult to identify relevant information.

- **Don't Rely Only on the Language Model**

The model should use retrieved information rather than depending entirely on its internal knowledge.

- **Don’t Ignore Hallucinations**

  Generated responses should be checked against retrieved sources to avoid unsupported information.

- **Don’t Store Sensitive Data Without Protection**

  Private and confidential information should be secured using proper access controls and security measures.

- **Don’t Depend Only on Keyword Search**

  Semantic search using embeddings generally provides better retrieval results than keyword matching alone.

- **Don’t Overload the Prompt**

  Providing excessive context can increase costs and reduce response quality.

- **Don’t Skip Testing**

  Regular testing helps identify retrieval issues and maintain system reliability.

2.3. LIMITATIONS OF RAG SYSTEMS

Although RAG systems improve response quality, they still have certain limitations.

- **Dependency on Data Quality**

  The quality of responses depends heavily on the quality of the available documents.

- **Retrieval Errors**

  The retrieval system may sometimes fail to identify the most relevant information.

- **Hallucinations**

  The language model may still generate unsupported content despite having retrieved information.

- **Complex Implementation**

  Building a RAG system requires document processing, embeddings, vector databases, and retrieval mechanisms.

- **Performance Challenges**

  Large document collections can increase storage requirements and retrieval time.

- **Maintenance Requirements**

  Knowledge bases must be updated regularly to keep information current.

- **Limited Reasoning**

  RAG improves access to information but does not guarantee perfect reasoning for complex problems.

- **Security and Privacy Concerns**

  Sensitive information must be protected to prevent unauthorized access.

---

3. EMBEDDING MODELS AND VECTOR DATABASES

- Embedding models convert text into numerical vectors that represent semantic meaning. Similar pieces of information produce similar vectors, allowing the system to identify related content.
- Vector databases store these embeddings and perform similarity searches to find the most relevant information for a user's query. Popular vector databases include FAISS, ChromaDB, Pinecone, Qdrant, and Weaviate.
- Together, embedding models and vector databases form the foundation of the retrieval process in RAG systems.

---

4. WHAT I LEARNED TODAY

- Learned the best practices for building effective RAG systems.
- Understood common mistakes that should be avoided in RAG applications.
- Learned the limitations and challenges of RAG systems.
- Understood the purpose of embedding models and vector databases.
- Learned how similarity search helps retrieve relevant information.
- Understood how data quality affects response accuracy.

---

5. OUTCOME

Understood the key concepts related to RAG systems, including best practices, limitations, embeddings, and vector databases. Also learned how these components work together to improve information retrieval and response generation.

---

6. CONCLUSION

RAG systems improve the quality of AI-generated responses by combining information retrieval with language generation. Proper document management, embeddings, vector databases, and retrieval techniques play an important role in building reliable RAG applications. Understanding both the strengths and limitations of RAG systems is essential for developing effective AI solutions.